

# Lesson 1: How the Web Works (Browsers, Servers, HTML)

## Mission Brief

Cadet, before you can build starships, you need to understand the space they fly through. Every website you've ever visited traveled across a vast network of cables, satellites, and servers to reach your screen in milliseconds. Today you become a Lumexa Navigator: you'll trace the exact path a web page takes from a faraway server to your browser, and write your first line of HTML — the language every page in the galaxy is built from.

## Learning Objectives

- Explain what happens, step by step, when a browser requests a web page
- Define "client," "server," "HTTP request," and "HTTP response" in your own words
- Identify the role of a URL and its parts (protocol, domain, path)
- Write and save a valid, minimal HTML document
- Open an HTML file in a browser and view it using DevTools

## What You'll Build

A single file called `mission-log.html` — your first working web page, containing a heading, a paragraph, and a comment explaining what each line does.

## Prerequisites

None — this is the first lesson of the course. General computer literacy (saving files, using a text editor) is assumed.

## Key Concepts

- Client and server
- HTTP request / HTTP response
- URL (protocol, domain, path)
- DNS (Domain Name System)
- HTML (HyperText Markup Language)
- Rendering engine

- DevTools

## Concept Explanation

Every time you type a web address into your browser, you kick off a small but remarkable journey. Your computer (the **client**) doesn't already have the page sitting inside it — it has to ask another computer somewhere else in the world (the **server**) to send it over. This back-and-forth is called the **client-server model**, and it's the foundation of literally everything on the web.

Here's the sequence: you type `https://lumexa.space/missions` into your address bar. Your browser first has to figure out *which* computer on the internet owns `lumexa.space`. It asks the **DNS (Domain Name System)** — think of DNS as the galaxy's phone book, translating human-friendly names like `lumexa.space` into a numeric **IP address** like `104.21.55.12`, which is the actual "location" of the server on the network.

Once your browser knows the address, it sends an **HTTP request** to that server. HTTP stands for HyperText Transfer Protocol — it's just an agreed-upon format for "asking for things" and "sending things back." The request basically says: "Hey, can I have the file at `/missions`?" The server receives this, finds (or generates) the right file, and sends back an **HTTP response**, which includes a status code (like `200 OK` for success or `404 Not Found` for a missing page) and the actual content — usually an **HTML** document.

HTML, or HyperText Markup Language, is not a programming language — it's a **markup language**. That means it doesn't calculate or make decisions; it describes the *structure* of content using **elements**, written as tags like `<p>` for a paragraph or `<h1>` for a heading. Your browser's **rendering engine** reads this HTML from top to bottom and turns it into the visual page you see — text, boxes, images, all positioned according to the markup and, later in this course, CSS.

Let's break down a URL, since you'll type these constantly: `https://lumexa.space/missions/mars` - `https://` is the **protocol** — the rules for how data is transferred securely (the "s" means encrypted). - `lumexa.space` is the **domain** — the human-readable server name. - `/missions/mars` is the **path** — which specific resource on that server you want.

Finally, every modern browser ships with **DevTools**, a built-in set of inspection tools. You can open them with `F12` or right-click → "Inspect." The **Elements** panel shows you the live HTML structure of any page, and the **Network** panel shows you every request and response happening behind the scenes — you'll use this constantly throughout the course to debug your work.

## Guided Coding

```

<!-- mission-log.html -->
<!-- The doctype tells the browser "this is a modern HTML5 document" -->
<!DOCTYPE html>
<html lang="en">
  <head>
    <!-- The head holds metadata: information ABOUT the page, not shown on the page itself -->
    <meta charset="UTF-8" />
    <meta name="viewport" content="width=device-width, initial-scale=1.0" />
    <title>My First Mission Log</title>
  </head>
  <body>
    <!-- Everything inside body IS the visible page content -->
    <h1>Cadet Mission Log – Entry 01</h1>
    <p>
      Today I learned how a web page travels from a server to my browser.
      My first HTML page is now live on my own computer.
    </p>
  </body>
</html>

```

## Code Walkthrough

- `<!DOCTYPE html>` — Must be the very first line. It tells the browser to use modern HTML5 rendering rules instead of an old compatibility mode.
- `<html lang="en">` — The root element that wraps the entire document. `lang="en"` tells browsers and screen readers the page's language.
- `<head>` — A container for metadata: the page's `<title>`, character encoding, and viewport settings. Nothing inside `<head>` is displayed directly in the page body.
- `<meta charset="UTF-8" />` — Declares the text encoding so special characters (emoji, accented letters) display correctly.
- `<meta name="viewport" ...>` — Tells mobile browsers to size the page to the device's screen width instead of shrinking it down. You'll rely on this heavily in the responsive design lesson.
- `<title>` — The text shown in the browser tab.
- `<body>` — Everything a visitor actually sees goes here.
- `<h1>` — The top-level heading, used once per page for the main title.
- `<p>` — A paragraph of regular text.

## Student Mission

Create `mission-log.html` exactly as shown above, but personalize it: 1. Change the heading to include your own callsign (a fun made-up name). 2. Add a second `<p>` describing one thing you find interesting about how the internet works. 3. Add an HTML comment above your second paragraph explaining, in your own words, what a

"server" is. 4. Open the file directly in your browser (double-click it, or drag it into a browser window) and confirm it displays correctly.

## Challenge

Add a third paragraph that lists, in order, the four steps that happen between typing a URL and seeing a page (DNS lookup → HTTP request → HTTP response → rendering). Then open DevTools, go to the **Network** tab, refresh a real website (like `example.com`), and find the actual HTTP status code returned for the main document. Write that status code as a comment in your file.

## Experiment/Extension

Try renaming your file's `<title>` to something else and reload the page — watch the browser tab text update instantly. Then try deliberately misspelling `<!DOCTYPE html>` as `<!DOCTYPE htm1>` and reload — most browsers will still render the page, but note in a comment what "quirks mode" means and why avoiding it matters (search: "browser quirks mode").

## Common Mistakes

- Forgetting the closing tag (e.g. `<p>` without `</p>`), which can cause later content to nest incorrectly.
- Placing visible content directly inside `<head>` instead of `<body>`.
- Saving the file with the wrong extension (e.g. `.txt` instead of `.html`), so the browser doesn't recognize it as a web page.
- Forgetting `<!DOCTYPE html>`, which can trigger inconsistent rendering across browsers.
- Confusing "the internet" (the physical/network infrastructure) with "the web" (HTTP + HTML, one of many things that run on the internet).

## Debugging Tips

- If your page shows raw text instead of a formatted page, double-check the file extension is `.html`, not `.html.txt` (some editors hide extensions by default).
- Open DevTools ( `F12` or right-click → Inspect) and check the **Console** tab for red error messages — for plain HTML there usually won't be any yet, but it's a habit you'll need soon.
- Use the **Elements** panel to click around your rendered page and see exactly which tag produced which piece of content.
- If a tag isn't behaving as expected, check for a missing closing tag by looking at how your editor auto-indents — misaligned indentation is often a sign of a missing `</tag>`.

## Check Your Understanding

1. What is the difference between a client and a server?
2. What does DNS do, and why is it necessary?
3. What are the three parts of the URL `https://lumexa.space/missions` ?
4. Why is HTML called a "markup language" rather than a "programming language"?
5. What is the purpose of the `<head>` section of an HTML document?

## Mini Quiz + Answer Key

**Quiz** 1. Which HTTP status code typically means "page not found"? a) 200 b) 301 c) 404 d) 500 2. True or False: The `<title>` tag's content is displayed inside the page body. 3. What does the `s` in `https` stand for? 4. Which tag must be the very first line of every HTML5 document? 5. What panel in DevTools shows live network requests?

**Answer Key** 1. c) 404 2. False — it appears in the browser tab, not the body. 3. Secure (encrypted communication via SSL/TLS) 4. `<!DOCTYPE html>` 5. The Network panel

## Lesson Recap

- The web works on a client-server model: your browser requests, a server responds.
- DNS translates domain names into IP addresses.
- HTTP requests and responses carry data between client and server.
- HTML structures content using nested elements/tags; it doesn't compute logic.
- DevTools let you inspect both the rendered structure (Elements) and the network traffic (Network) of any page.

## Homework

Write a short "mission report" (3-5 sentences, as a new paragraph in your `mission-log.html` file) explaining, in your own words and without copying the lesson text, what happens between you pressing Enter after typing a URL and the page appearing on screen.

## Portfolio Project Connection

This lesson lays the foundation for **all three course projects** — every project starts with a valid `<!DOCTYPE html>` document with a correct `<head>` and `<body>` . You'll use this exact document skeleton as the starting point for `index.html` in Project 01 (Personal landing page).